

Chapter 4

Structures

by Lorraine Gaudio

Version May 29, 2026

Contents

1	Overview	1
2	R Notebooks—Part 2	2
2.1	Default Text	3
3	Transparency	3
3.1	Indexing Vectors	4
4	Logical Vectors	6
4.1	Comparison Operators	6
4.2	Logical Indexing	7
5	Summary	8
6	Chapter Terms	8
7	Practice Space	10
7.1	Task 1	10
7.2	Task 2	11
7.3	Task 3	11
7.4	Task 4	11
7.5	Task 5	12
7.6	Task 6	12
7.7	Task 7	12
8	References	12

1 Overview

Now that you can work in an R Notebook and write usable memos, Chapter 4 raises the standard for documentation: transparency.

In research, correct code is not enough. You also need to document where help came from, what you learned from it, and how you checked your result. This helps you show your learning honestly, return to your work later, and make your analysis understandable to someone else.

This chapter also finishes the vector-selection skills you need before working with larger datasets. You will learn how vector elements can be labeled with names, how indexing lets you select specific values, and how logical values can be used to filter a vector.

The point of this chapter is control: selecting the values you mean to select, checking whether your selection worked, and documenting your process clearly enough that someone else could follow it.

By the end of Chapter 4, you should be able to practice transparency in your notebook memos, select vector elements by position and name, build logical vectors using comparison operators, and use logical indexing to filter values from a vector.

In Chapter Four, you will practice how to:

- Practice transparency in code memos by documenting goals, checks, explanations, and outside resources.
 - Create named vectors using element names.
 - Use indexing to select vector elements by position.
 - Use indexing to select vector elements by name.
 - Recognize that character indexing requires exact matching and returns NA when a name is not found.
 - Build logical vectors using TRUE, FALSE, and NA.
 - Use comparison operators such as >, <, ==, >=, <=, and != to create logical results.
 - Use logical indexing to filter values from a vector.
 - Verify that your selected or filtered vector contains the expected values.
-

2 R Notebooks—Part 2

In Chapter 3, you learned how to use an R Notebook (`.Rmd`) as a “reproducible thinking” document: code + output + memo-style writing in one place, in order. This section is a quick setup review before you begin the Chapter 4 work.

Here’s what you can do now:

1. **Create and save an R Notebook** (File > New File > R Notebook) and store it in your course folder.
2. **Recognize the YAML header** at the top of the file and edit the title.
3. **Insert code chunks** (Insert button, shortcut, or typing the fences) and understand that R code only runs inside chunks.
4. **Run chunks** using the play button, Run menu, or keyboard shortcuts and see output directly below.
5. **Memo correctly:**
 - Inside a code chunk: text must start with # otherwise R tries to run it.
 - Outside a code chunk: write normal paragraphs with no # needed.
 - Use the notebook to document your learning process, including your goal, checks, explanations, and resources used.

An R Notebook is only useful if a human can read it. In this chapter, you will keep building clean notebook habits: preserving the **YAML header**, organizing your work clearly, writing readable memos, and preparing a notebook that can knit successfully to HTML.

2.1 Default Text

Please open a new R Notebook in your course folder named `chapter4_notes.Rmd`.

When you create a new R Notebook or R Markdown file, RStudio provides a template. It is designed to demonstrate the three core components of a notebook:

- The YAML header: The block of text at the very top, enclosed between two lines of three dashes (---). This tells RStudio how to knit, or render, your document.
- Markdown Text: The white background text (like “This is an R Markdown Notebook”). This shows you how to write regular prose.
- Code Chunks: The gray boxes containing code (like `plot(cars)`). This shows you how to embed executable R code.

2.1.1 Clear the Default Notebook Body

To start fresh, delete all the default text in the R Notebook so you have a blank canvas to work with.

2.1.1.1 Keep the Header Do not delete the text at the very top between the — lines. This is your metadata (Title, Author, Output format). If you delete this, your file might not run correctly.

Your header should look something like this:

```
---
title: "Chapter 4"
output: html_notebook
---
```

Keep this top section. Delete the example text and code below it.

2.1.1.2 Delete the Body You can safely delete everything below the second --- line. Keep the YAML header at the top.

1. Place your cursor on the line immediately following the second ---.
2. Highlight everything all the way to the bottom of the document.
3. Press Delete or Backspace.

3 Transparency

Science is not just about numbers; it is also about the values that guide how and why we do our work. Researchers, policymakers, and the general public must be able to trust scientific expertise for this work to have meaning (Resnik & Elliott, 2023). Transparency in academic work promotes that trust. **Transparency** means disclosing methods, materials, assumptions, values, and interests so another person can understand what you did and reproduce it (Resnik & Elliott, 2023).

Transparency is associated with these three related values:

1. **Honesty** means reporting what actually happened, including shortcuts, mistakes, unexpected results, and outside help. You record what you tried even when it did not produce the result you expected. You state what you do not know yet. You do not present outside work as your own. If you use a peer, website, AI tool, instructor, or other resource, document what you used and what you learned.
2. **Openness** means making your work inspectable so someone else can see what you used and how you got your result. Reproducibility is a key part of openness. In this course, that means sharing the data source, code, steps, and explanations needed for someone else to follow your workflow.

3. **Accountability** means taking responsibility for your work. In this course, accountability means checking your results, correcting mistakes, responding to feedback, and documenting your process clearly enough that someone else can understand your choices.

In this course, **research journals** are running records of your R analysis process. They are internal documents that help you prepare reports, return to old work, and explain your decisions later.

In this course, your R Notebook works like a research journal. Add comments before or near your code explaining:

- Goal: What are you trying to do?
- Prediction: What do you expect the code to produce?
- Code/Test: What code did you use to test your idea?
- Record what you tried: What did you try, including code that did not work?
- Verify: How did you check whether the result was correct?
- Comment: What does the output mean?
- Reflect: What did you learn, and how could this help you later?
- Resource Cited: What resource did you use, and what did you learn from it?

You do not need every memo label for every small line of code. Use memo labels when your own code does something surprising, confusing, or important to check.

Next, you will practice these transparency habits while learning how R labels vector elements and selects specific values.

3.1 Indexing Vectors

The **indexing operator** `[ ]` is the standard way to extract parts of a vector. The basic pattern is `x[i]`, where `x` is the vector and `i` is the index, or indices, of the elements you want to extract. For the atomic vectors in this chapter, `[ ]` can select one element or many, and it returns an object of the same type as `x`.

An **index** is a value used to identify elements in an object. In R, indices are often integers for positions, character values for names, or logical values such as `TRUE` and `FALSE`.

3.1.1 Positional Indexing

Goal Use `[ ]` with index positions to extract values from a vector.

```
# Type this in your R Notebook
z <- c(10, 20, 30, 40, 50) # create a vector
```

```
# Type this in your R Notebook
z[1]
```

```
## [1] 10
```

This returns the first element by position.

```
# Type this in your R Notebook
z[3]
```

```
## [1] 30
```

This returns the third element by position.

```
# Type this in your R Notebook
z[2:4]
```

```
## [1] 20 30 40
```

This returns the second, third, and fourth elements by position.

Notice: `z[2:4]` selects positions 2 through 4. It does not search for the *values* 2, 3, and 4.

```
# Type this in your R Notebook
z[-c(1, 5)]
```

```
## [1] 20 30 40
```

Negative indexing removes elements. This returns all elements except the first and last.

Notice: R counts positions starting at 1, not 0. Also notice that negative positions remove values instead of selecting them.

3.1.2 Character Indexing

When a vector has names, you can use those names to index into the vector. **Character indexing** uses the names of the elements to retrieve specific values.

There are two different naming ideas here.

1. **Object name** is the variable name `nerdy_palette` that you use to refer to the entire vector.
2. **Element names** are the labels assigned to each element within the vector (e.g., "Octarine", "TARDIS Blue").

Element names can be used in indexing to access specific elements of the vector. A named vector works like a small lookup table: each value has a label, and you can use the label to retrieve the matching value.

Goal: Create a named vector of color codes, then use character indexing to return the hexadecimal color code for "Octarine".

```
# Create nerdy_palette
nerdy_palette <- c(
  "Octarine" = "#7F00FF",
  "TARDIS Blue" = "#003B6F",
  "Rebeccapurple" = "#663399",
  "Hoolooovoo" = "#4F97A3",
  "Cosmic Latte" = "#FF8E7",
  "Vantablack" = "#000000",
  "Matrix Green" = "#03A062",
  "Source Engine Magenta" = "#FF00FF",
  "Command Red" = "#CC0000",
  "Impossible Green" = "#4D9E68"
)

# Use character indexing to return the hex code for Octarine
nerdy_palette["Octarine"]

## Octarine
## "#7F00FF"
```

When you run this chunk, it should return the hexadecimal color code for "Octarine" from the `nerdy_palette` vector.

Character indexing requires exact matching on names. If you ask for a name that doesn't exist, you get NA for that element.

Use character indexing to retrieve the color codes for "Octarine," "TARDIS Blue," and "NotAColor" from the `nerdy_palette` vector.

```
# Type this in your R Notebook
nerdy_palette[c("Octarine", "TARDIS Blue", "NotAColor")]
```

```
##      Octarine TARDIS Blue      <NA>
##      "#7F00FF"  "#003B6F"      NA
```

The output should return the color codes for "Octarine" and "TARDIS Blue", plus NA for "NotAColor" because that name does not exist in the vector.

3.1.3 More Indexing by Position

Positional indexing uses the numeric positions of elements in the vector to retrieve specific values.

Use positional indexing to return the first and third color codes from the `nerdy_palette` vector.

```
# Type this in your R Notebook
nerdy_palette[c(1, 3)]
```

```
##      Octarine Rebeccapurple
##      "#7F00FF"  "#663399"
```

The output shows the hexadecimal color codes for Octarine and Rebeccapurple in the `nerdy_palette` vector.

We will return to indexing in later chapters when we learn about data frames.

4 Logical Vectors

Review from Chapter 2.

Logical vectors are used for conditions and comparisons. They contain TRUE or FALSE values.

Create a logical vector named `real` that contains the values TRUE, FALSE, and NA using the `c()` function.

```
# Type this in your R Notebook
real <- c(FALSE, TRUE, TRUE, FALSE, TRUE, TRUE, TRUE, TRUE, TRUE, NA)

# Check the real vector
print(real)
```

```
## [1] FALSE  TRUE  TRUE FALSE  TRUE  TRUE  TRUE  TRUE  TRUE  NA
```

The output prints the logical values stored in the `real` vector.

4.1 Comparison Operators

Logical vectors can also be created using comparison operators. These operators compare values and return TRUE or FALSE based on the comparison.

Comparison operators include:

- `>` greater than
- `<` less than
- `==` equal to

- `>=` greater than or equal to
- `<=` less than or equal to
- `!=` not equal to

Create a logical vector named `logicals` that contains the results of the following comparisons: `1 > 2`, `3 < 5`, and `4 == 4` using the `c()` function.

```
# Type this in your R Notebook
logicals <- c(1 > 2, 3 < 5, 4 == 4)

# Check the logicals vector
print(logicals)
```

```
## [1] FALSE TRUE TRUE
```

The output shows the results of the comparisons stored in the `logicals` vector. There is one `FALSE` value and two `TRUE` values. The first value is `FALSE` because 1 is not greater than 2. The second value is `TRUE` because 3 is less than 5. The final value is `TRUE` because 4 is equal to 4.

4.2 Logical Indexing

Logical values can be used to select elements from a vector. **Logical indexing** uses a logical vector inside `[ ]`. Values paired with `TRUE` are kept. Values paired with `FALSE` are left out.

Create a new vector called `smaller` that contains only the values in `number_list` that are less than or equal to 20.

```
# Type this in your R Notebook
number_list <- c(10, 25, 5, 30, 15, 40)
```

The `number_list` vector contains the values 10, 25, 5, 30, 15, and 40.

```
# Step 1: create the logical "mask"
mask <- number_list <= 20

# Check the mask
print(mask)
```

```
## [1] TRUE FALSE TRUE FALSE TRUE FALSE
```

This prints a logical vector (`TRUE/FALSE`) the same length as `number_list`. Each `TRUE` marks a value that is less than or equal to 20.

For logical indexing to behave as expected, the logical vector should line up with the vector you are filtering. Here, `mask` has one `TRUE` or `FALSE` value for each value in `number_list`.

```
# Step 2: use the logical vector inside [ ]
smaller <- number_list[mask]
print(smaller)
```

```
## [1] 10 5 15
```

This keeps only the values where `mask` is `TRUE`, so `smaller` contains 10, 5, and 15.

The key idea is that the condition `number_list <= 20` creates a logical vector, and `[ ]` uses that logical vector as a filter.

5 Summary

In this chapter, you strengthened your R Notebook habits by practicing transparency in your code memos. You learned to document your goals, explain what your code produced, verify your results, and record outside resources when you used help. These habits make your work easier to understand, check, revise, and share.

You also learned how to select values from vectors. You practiced positional indexing to select elements by number, negative indexing to remove elements, and character indexing to select elements from named vectors. You saw that character indexing requires exact matching and returns NA when a requested name does not exist.

Finally, you learned how logical vectors support filtering. You created logical values with comparison operators, stored those results in a logical vector, and used logical indexing to keep only the values that met a condition.

Your main takeaway is that transparent notebooks make your coding choices inspectable, and indexing gives you control over which vector values you select or filter.

6 Chapter Terms

Accountability: Taking responsibility for the results you report by verifying them, responding to questions or critiques, and correcting errors when found. In practice, this means you keep a clear record (e.g., in an R Markdown/Notebook or script) that shows how you checked your work and why you made your choices.

Backtick: The ``` character (above the Tab key) used for code-related formatting and quoting. In **Markdown**, single backticks mark *inline code* and triple backticks create *fenced code blocks*; in **R Markdown**, backticks also mark *inline R code* (``r ...``), including inside YAML fields like date: “``r Sys.Date()``”, which gets evaluated when you render the document. In R, backticks can also quote non-syntactic names (e.g., names with spaces) so they can be used safely in code.

Character Indexing: Indexing a named vector using element names (character strings) instead of numbers. Names must match exactly; a missing name returns NA.

Character String: A single piece of text in R, stored as a character value and written in quotes (for example, “My favorite color is yellow!” or “Who stole the cookie from the cookie jar?”).

Class `class()`: A function that reports the object’s class, which tells R how to interpret the object and which methods to use when functions are applied to it. For example, `class(3.14)` returns “**numeric**”, and `class(mtcars)` returns “**data.frame**”.

Code Chunks: Fenced blocks in an R Markdown/Notebook document where R code is written and executed, usually starting with “`{r}`”. Only code inside a chunk runs, and its output (including plots and tables) appears directly below the chunk when executed.

Element Names: The individual labels assigned to elements inside a named vector (for example, “Octarine” or “TARDIS Blue”). Element names let you index by label (e.g., `nerdy_palette["Octarine"]`) to retrieve specific values.

Factor Class: An R object class for categorical data, where each value is stored as an integer code that points to a category label. Factors help R handle categories correctly in tables, plots, and statistical models, even though they may print like text.

Factor Levels: The distinct category labels a factor can take (for example, “**freshman**”, “**sophomore**”, “**junior**”, “**senior**”). Levels define the set and order of categories, and each factor value is internally stored as the index of its level.

Honesty: Accurately reporting what you did in R—your actual steps, mistakes, revisions, and uncertainty—without presenting outside work as your own. This includes clearly attributing code, ideas, and any AI assistance, and being able to explain the code you submit.

Indexing Operator: The square-bracket indexing operator `[` or `[[` are used to extract or replace parts of an object in R (for example, `x[i]`). For vectors, it selects one or more elements and returns the same kind of object as `x` (a vector).

Logical Operators: Operators that create or combine logical elements. The most common are comparison operators—`==`, `!=`, `<`, `>`, `<=`, `>=`—which return `TRUE/FALSE`, and Boolean operators—`!` (NOT), `&` / `|` (AND/OR applied element-by-element), and `&&` / `||` (AND/OR that evaluate only the first element, mainly for if conditions).

Logical Vector: A vector that stores Boolean elements: `TRUE` or `FALSE` (and sometimes `NA` for missing). Logical vectors are commonly produced by comparisons (for example, `x > 0`) and are heavily used for filtering and conditional code.

NA: A special constant that marks a missing value (“Not Available”)—meaning the data point is unknown or not recorded. R also provides typed missing values like `NA_integer_`, `NA_real_`, `NA_character_`, and `NA_complex_` to match the underlying data type.

Named Vectors: Vectors that have labels attached to their elements via the `names()` attribute, so you can refer to values by name instead of only by position. Named vectors are useful as simple lookup tables (for example, retrieving a hex code by a color name).

Object Name: The variable name you assign with `<-` that refers to the entire object in your environment. You use the object name to print, modify, or pass the whole object to functions.

Openness: Making your analysis inspectable and reproducible by sharing the inputs and decisions that produced your results (data source, packages, code, and key assumptions). In this course, openness means you provide others with enough details so they are able to rerun your work and get the same outputs.

Package: An installable collection of R code—functions, documentation, and sometimes datasets—that adds new features beyond base R. Packages are commonly installed from CRAN and then loaded with `library()` so you can use their tools in your session.

Positional Indexing: Indexing a vector using numeric positions (for example, `x[1]` for the first element or `x[c(1, 3)]` for the first and third). Positions are based on the current order of the vector.

R Markdown: A document format (`.Rmd`) that mixes plain-text writing (Markdown) with embedded R code chunks, so you can generate reports that combine narrative, code, and computed output. You can render an R Markdown file into finished formats like HTML, PDF, or Word.

R Notebook: An `.Rmd` (R Markdown) document that lets you run code in chunks and see the results (output, tables, plots) immediately below each chunk. It’s designed for reproducible work because it keeps your code, results, and explanatory notes together in the same file.

Research Journal: A running record of your R analysis process. This includes what question you asked, what code you tried (including failed attempts), what outputs you got, and what decisions you made along the way. In practice, it’s maintained in an R script or R Markdown/Notebook with dated notes, comments, and citations so you (or someone else) can retrace, verify, and reproduce the work.

Transparency: The practice of clearly disclosing your methods, materials, assumptions, tools, values, and sources so another person can follow your workflow, reproduce your results, and evaluate your claims. In this course, transparency supports explanatory power (you can explain what your code does and why) and originality/accountability (you produce and attribute work honestly, including documenting if and how you used AI).

Vectorization: R’s behavior of applying a function to an entire vector at once, producing results element-by-element without writing an explicit loop. For example, `paste(first, last)` combines the first elements together, then the second, then the third, returning a vector of full names.

YAML header: The metadata section at the top of an R Markdown or Quarto document, written between `---` lines, that controls document settings like the title, author, date, output format (HTML/PDF/Word), and options for rendering.

7 Practice Space

Fill in the blanks to complete the code in your R Notebook.

Before you begin:

1. Create a new R Notebook in RStudio.
2. Save it in your course folder as `chapter4_practice.Rmd`.
3. Complete all Practice Space tasks in that notebook.
4. Save your workspace as `chapter4_practice.RData`.

Required files to submit:

- `chapter4_practice.Rmd`
- `chapter4_practice.RData`

As you work, add short comments or memo notes explaining what your code is doing and what the output shows.

- **Add Comments** asks you to explain what the code or output shows.
- **Reflect** asks you to explain your thinking process, what confused you, what changed in your understanding, or what strategy you would use next.

These are there to help you practice the habit of writing memos as you work. Keep responses short: 1–3 thoughtful sentences is enough. For this Practice Space, answer the **Add Comments** prompt for each coding task. Then answer **Reflect** prompts where your thinking, prediction, confusion, or troubleshooting changed.

If you encounter an error, document your **tenacity** by briefly noting what happened, what you tried, and how you fixed it or what still confuses you. Document how you troubleshooted the error and what resource(s) you used to help you resolve the issue (**transparency**).

You may ask an AI to explain an error or provide you with more details to a curiosity. Do not ask AI to rewrite or complete any part of the report for you. You may quote AI output only when you clearly mark it as quoted text, either with quotation marks or a block quote using `>`. Then hand-type your own explanation of why you are including that quote.

Before you finish, clear your Environment and run the notebook from top to bottom. Then knit to HTML to check that the notebook works in order.

7.1 Task 1

Create a named vector

Practice creating a named vector by pairing values with element names. Some labels and some values are missing. The blanks test both parts of the name-value pair. **Tip:** Include quotes around the element names.

Create a named vector called `student_counts` that stores the number of students in four campus groups. Use the exact group names and counts: "First Year" count is 18, "Sophomore" count is 22, "Junior" count is 16, and "Senior" count is 14.

```
# Type this in your R Notebook. Fill in the blanks.
student_counts <- c(
  _____ = 18,
  "Sophomore" = ___,
  _____ = 16,
  "Senior" = ___
)

# Check the vector
print(student_counts)
```

Add Comments: In 1–3 sentences, explain what `print(student_counts)` shows. Include how the group names help you understand what each number means.

Reflect: Which part of this task was easier to keep track of: the values or the names? Explain how you checked that each group name matched the correct count.

```
missing element names: "First Year", "Junior"
missing values: 22, 14
```

7.2 Task 2

Use negative indexing

Practice using negative positions inside `[ ]` to remove elements from a vector.

Use negative indexing (`c()` with a negative sign before it) to remove the 1st and 5th values from the `quiz_scores` vector.

```
# Type this in your R Notebook.
quiz_scores <- c(85, 92, 78, 88, 61, 95)

# Fill in the blank to remove positions 1 and 5.
quiz_scores[____(1, 5)]
```

Add Comments: In 1–3 sentences, explain which quiz scores were removed and which scores stayed in the output.

Negative indexing removed _____.

Reflect: How did you know that 1 and 5 meant positions, not score values? Explain what you would check if the output removed the wrong scores.

7.3 Task 3

Use character indexing

Practice selecting a named vector element using an exact character name.

Use character indexing to retrieve the number of "Junior" students from the `student_counts` vector you created in Task 1.

```
# Fill in the blank with the exact element name.
____["_____"]
```

Add Comments: In 1–3 sentences, explain what the code returns. Include both the name and the value shown in the output.

Reflect: Does the exact spelling of "Junior" matter here? Why? Explain what you would check if R returned NA or did not find the value.

7.4 Task 4

Create a logical vector

Practice using a comparison operator to create a logical vector.

Create a logical vector called `meets_goal` that checks whether each value in `quiz_scores` vector from Task 2 is greater than or equal to 80.

```
# Use the quiz_scores vector from Task 2.

# Fill in the blank with the correct comparison operator.
```

```
meets_goal <- quiz_scores ____ 80

# Check the logical vector
print(meets_goal)
```

Add Comments: In 1–3 sentences, explain what each TRUE and FALSE means in `meets_goal`. Be specific about the cutoff value of 80.

``TRUE`` means _____. ``FALSE`` means _____.

Reflect: Before running the code, how many TRUE values did you expect? Explain how you compared your prediction to the printed output.

7.5 Task 5

Use logical indexing

Practice using a logical vector as a mask to filter another vector.

Use `meets_goal` from Task 4 to keep only the values in `quiz_scores` that are greater than or equal to 80.

```
# Use the logical vector from Task 4 to filter quiz_scores.
passing_scores <- ____[____]

# Check the new vector
print(passing_scores)
```

Add Comments: In 1–3 sentences, explain which scores were kept in `passing_scores` and why those scores matched the `meets_goal` mask.

``passing_scores`` kept _____ because _____.

Reflect: Explain how the logical vector worked like a filter. What would happen if `meets_goal` had FALSE in a position where you expected TRUE?

7.6 Task 6

Reflect on logical masks

In 1–3 sentences, explain how the logical mask controls which values are kept from `quiz_scores`. Use the words TRUE, FALSE, and `position` in your answer.

7.7 Task 7

Final checks before submitting

Step 1: Sweep your Environment (start clean).

Click the **broom icon** in the Environment pane to clear objects.

Step 2: Run everything top-to-bottom.

Use **Run** → **Run All** to execute every chunk in order.

Step 3: Save and submit.

After your notebook runs without errors, save your notebook (`chapter4_practice.Rmd`) and save your workspace (`chapter4_practice.RData`) for submission to Canvas.

8 References

Resnik, D. B., & Elliott, K. C. (2023). Science, values, and the new demarcation problem. *Journal for General Philosophy of Science*, 54(2), 259-286.

Thomas, P. A. (2023). Wikipedia and large language models: perfect pairing or perfect storm?. Library Hi Tech News, 40(10), 6-8.